Maxwell_Penalty:
A 3D Electromagnetic Solver with SBP-SAT Penalty Boundaries

SMS 751 Lecture Code

Contents

Abstract

This document describes `Maxwell_Penalty`, a three-dimensional finite-difference solver for Maxwell's equations in a linear dispersive dielectric, coupled to a hyperbolic constraint-damping system. The spatial discretization uses fourth-order summation-by-parts (SBP) operators with one-sided closures at every physical face. Boundary conditions are imposed weakly via simultaneous approximation terms (SAT) — the penalty method — on all six Cartesian faces of the computational domain. The time integrator is classical fourth-order Runge–Kutta. The code includes two analytic sources — a plane wave with a compactly supported envelope, and a paraxial linearly-polarised Gaussian beam — and supports an inhomogeneous-permittivity region that acts as a lens through which the beam propagates. This document covers the physics, the numerical method, the penalty-boundary treatment, the source models and their limitations, and how to build, configure, and run the code.

1 Introduction: the physics

The code evolves the electromagnetic field $(\mathbf{E}, \mathbf{B})$ in a linear, possibly inhomogeneous, dielectric. Let $\varepsilon(\mathbf{x})$ be the permittivity, $\mu(\mathbf{x})$ the permeability, and $\sigma(\mathbf{x})$ the conductivity (Ohmic damping). Define $\mathbf{D} = \varepsilon\mathbf{E}$ and $\mathbf{H} = \mathbf{B}/\mu$. In Gaussian units, Maxwell's equations are

$$\partial_t \mathbf{D} = \nabla \times \mathbf{H} - 4\pi \mathbf{J}, \tag{1}$$
$$\partial_t \mathbf{B} = -\nabla \times \mathbf{E}, \tag{2}$$
$$\nabla \cdot \mathbf{D} = 4\pi\rho, \tag{3}$$
$$\nabla \cdot \mathbf{B} = 0, \tag{4}$$

with Ohm's law $\mathbf{J} = \sigma\mathbf{E}$ for the current density. Equations (1)–(2) are the time-evolution (*hyperbolic*) equations. Equations (3)–(4) are the divergence constraints. In the continuum these constraints are preserved by the time evolution: if $\mathbf{D}, \mathbf{B}$ initially satisfy them, they satisfy them for all time.

Numerically this is not automatic. Round-off error and truncation error in the discretized curl operators can seed small constraint violations that, in a naive scheme, grow without bound. The standard remedy, following Munz, Omnes, Schneider, and others, is to *extend the system* with two auxiliary scalar fields Ψ_D and Ψ_B that act as Lagrange multipliers for the constraints. The extended system (in vacuum form, generalized below) is

$$\partial_t \mathbf{D} - c^2 \nabla \Psi_D = \nabla \times \mathbf{H} - 4\pi \mathbf{J}, \tag{5}$$

$$\partial_t \mathbf{B} - c^2 \nabla \Psi_B = -\nabla \times \mathbf{E}, \tag{6}$$

$$\partial_t \Psi_D = \nabla \cdot \mathbf{D} - 4\pi\rho - \kappa_D \Psi_D, \tag{7}$$

$$\partial_t \Psi_B = \nabla \cdot \mathbf{B} - \kappa_B \Psi_B. \tag{8}$$

At $\Psi_D = \Psi_B = 0$ the original Maxwell system is recovered. For $\kappa_D, \kappa_B > 0$ the Ψ fields satisfy damped wave equations whose source is the constraint violation — any residual violation radiates outward at speed c and is exponentially damped on timescale $1/\kappa$. This absorbs round-off and truncation noise, and absorbs the $O(1/(kw_0)^2)$ constraint residual of the paraxial beam source (Sec. 4.2).

The code evolves nine fields: $(D_x, D_y, D_z, B_x, B_y, B_z, \Psi_D, \Psi_B, \rho)$. The charge density ρ is carried as an evolved variable for generality, driven by $\partial_t \rho + \nabla \cdot \mathbf{J} = 0$; for the simulations shipped with the code, ρ and $\mathbf{J}$ are initially zero and remain zero up to round-off.

The material parameters ε, μ, σ are stored as auxiliary gridfunctions in the local inverse form $\texttt{ieps} = 1/\varepsilon$, $\texttt{imu} = 1/\mu$, $\texttt{sigma} = \sigma$. They are spatially varying in general; the default configuration places an elliptical-contour inhomogeneity in ε to model a lens.

1.1 The flux form

The spatial derivatives in the extended system can be written as a quasi-linear hyperbolic system

$$\partial_t U + A_x \partial_x U + A_y \partial_y U + A_z \partial_z U = S(U, \mathbf{x}), \tag{9}$$

where $U = (D_x, D_y, D_z, B_x, B_y, B_z, \Psi_D, \Psi_B, \rho)^\top$ and S collects the source terms (conductivity damping, constraint-damping relaxation, and material-gradient couplings). The three flux matrices A_x, A_y, A_z encode the spatial propagation. Their characteristic decomposition is what the penalty method needs; we return to this in Sec. 3.

2 Spatial discretization

2.1 Grid, decomposition, and indexing

The domain is a rectangular parallelepiped $[x_0, x_n] \times [y_0, y_n] \times [z_0, z_n]$, discretized on a uniform Cartesian grid of global size (N_x, N_y, N_z) with spacings dx, dy, dz. The grid is distributed across MPI ranks via a Cartesian topology chosen automatically by greedy prime factorization. Each rank owns a contiguous block and is surrounded by ghost zones of width g_s (default $g_s = 3$). For physical boundaries the ghost zones are absent and the rank stores real data at every index; for MPI-internal and periodic boundaries, the ghost zones are synchronized via non-blocking point-to-point communication before each RK stage.

The local index into a 3D field f at grid point (i, j, k) is $\texttt{ijk} = i + jN_x + kN_xN_y$, with i the fastest-varying index. Strides are $d_i = 1$, $d_j = N_x$, $d_k = N_xN_y$, and every finite-difference stencil takes a stride argument so it can operate along any axis.

2.2 Summation-by-parts (SBP) operators

The spatial derivatives are computed by summation-by-parts finite differences of order $(4, 2)$ — fourth-order in the interior, second-order one-sided closures at the first four rows on each boundary. A $(4, 2)$ SBP operator D approximates d/dx on a grid of N points and factorizes as

$$D = H^{-1}Q, \qquad Q + Q^{\top} = B = \mathrm{diag}(-1, 0, \ldots, 0, +1), \tag{10}$$

where H is a positive-definite diagonal weight matrix (the "norm"), and Q is almost antisymmetric — the asymmetry captured entirely by the boundary term B. In the $(4, 2)$ case

$$H = h \cdot \mathrm{diag}(17/48,\ 59/48,\ 43/48,\ 49/48,\ 1, 1, \ldots), \tag{11}$$

with h the grid spacing. The key stability-enabling identity is

$$\langle u, Dv\rangle_H + \langle Du, v\rangle_H = u_N v_N - u_0 v_0, \tag{12}$$

where $\langle\cdot,\cdot\rangle_H = \sum_i H_{ii} u_i v_i$ is the discrete H-inner product. This is the discrete analogue of integration by parts on a finite interval — the boundary terms appear exactly, without truncation error, and it is this identity that lets us prove a discrete energy estimate for the scheme.

The interior stencil (rows $i = 4 \ldots N - 5$) is the standard fourth-order centered difference

$$(Du)_i = \frac{u_{i-2} - 8u_{i-1} + 8u_{i+1} - u_{i+2}}{12h}. \tag{13}$$

The four near-boundary rows use one-sided Strand (1994) closures; at the lower boundary:

$$\begin{aligned}
(Du)_0 &= \frac{-24u_0 + 59u_1 - 8u_2 - 3u_3}{17h}, \\
(Du)_1 &= \frac{-u_0 + u_2}{2h}, \\
(Du)_2 &= \frac{4u_0 - 59u_1 + 59u_3 - 4u_4}{86h}, \\
(Du)_3 &= \frac{3u_0 - 59u_2 + 64u_4 - 8u_5}{98h},
\end{aligned}$$

and symmetric closures at the upper boundary. These are implemented as `SBP42_L0`, `SBP42_L1`, `SBP42_L2`, `SBP42_L3` and their upper-boundary counterparts (`SBP42_RN`, etc.) in `derivatives.h`.

2.3 Runge–Kutta time integration

Time integration is classical explicit fourth-order Runge–Kutta. The timestep is chosen by the Courant–Friedrichs–Lewy condition

$$dt = \mathrm{cfl} \cdot \min(dx, dy, dz). \tag{14}$$

For the pure SBP interior (no SAT), stability permits $\mathrm{cfl} \lesssim 1$. With SAT penalties on all six faces the effective spectral radius increases near corners where multiple penalties stack, and the usable CFL drops to ≈ 0.2 in practice (Sec. 5).

3 Boundary conditions: the SBP-SAT penalty method

3.1 Why weak imposition

The naïve approach to boundary conditions — "overwrite the evolution at boundary nodes with the prescribed boundary value" — is called *strong imposition*. It is simple and often works, but it breaks the SBP energy estimate: a boundary row whose equation is replaced loses the clean integration-by-parts structure, and stability must be argued separately, usually by numerical experiment.

The *simultaneous approximation term* (SAT) method imposes boundary conditions *weakly*, as an additive penalty. The PDE is retained at every row, including the boundary row, and a correction term is *added* that pulls the solution toward the prescribed boundary value. The SAT term is tuned so that the full semi-discrete system admits a discrete energy estimate: summing the equations against $U^\top H$ and using the SBP identity produces boundary terms that are exactly cancelled by the SAT penalty. This gives provable stability for any $\tau \geq 1/2$ (see below), independent of the particular PDE coefficients or grid spacing.

The tradeoff is summarised below.

Property	Strong imposition	SBP-SAT (weak)
Boundary equation	Replaced entirely	Interior RHS + penalty
Energy estimate	No formal proof	Yes, provable for $\tau \geq 1/2$
Stability w/ non-uniform ε, μ	No guarantee	Provable
Convergence order	Same as interior closures	Same as interior closures
Penalty scaling	N/A	$\tau \cdot H_{00}^{-1}/h$

Both methods give the same formal convergence order — the boundary truncation error is set by the SBP one-sided closure, not by how the boundary condition is imposed. The pedagogical reason to prefer SBP-SAT is the discrete energy estimate, which extends to variable coefficients, multi-dimensional grids, and characteristic couplings (see Sec. 3.6) — all places where a strong imposition would need a separate stability argument.

3.2 Characteristic decomposition

The penalty acts on the *incoming* characteristic at each face. For a hyperbolic system $\partial_t U + A\partial_x U = 0$ with flux matrix A, diagonalize

$$A = V\Lambda V^{-1}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n), \tag{15}$$

and split into positive and negative eigenvalue parts

$$A^+ = V\Lambda^+V^{-1}, \qquad A^- = V\Lambda^-V^{-1}, \qquad \Lambda^\pm = \mathrm{diag}(\max(\pm\lambda_i, 0)). \tag{16}$$

The characteristic variables $W = V^{-1}U$ satisfy $(\partial_t + \lambda_i\partial_x)W_i = 0$; modes with $\lambda_i > 0$ propagate in $+x$ (incoming at the lower face $x = 0$), modes with $\lambda_i < 0$ propagate in $-x$ (incoming at the upper face $x = L$).

At the lower-x face, we prescribe the *incoming* characteristics (the $+\lambda$ modes); the $-\lambda$ modes are outflow and are determined by the interior dynamics. At the upper-x face, the roles are reversed.

3.3 The penalty form

At the lower-x boundary (grid index $i = 0$), the SAT correction applied to each variable is

$$\dot{U}_0 \ \ -= \ \tau \cdot \frac{(H^{-1})_{00}}{h} \cdot A^+ \cdot (U_0 - g), \tag{17}$$

where g is the prescribed boundary state. The scalar $\tau \geq 1/2$ is the penalty strength; the code uses $\tau = 1$ by default. $(H^{-1})_{00} = 48/17$ for the $(4, 2)$ SBP norm. The stencil operates only on the single boundary row — rows $i = 1, 2, 3$ receive the SBP one-sided derivative (Sec. 2.2) but *no* additional SAT term; they contribute to the energy argument through the norm integral.

The analogous correction at the upper boundary uses A^- (or, equivalently, the positive-definite projection $|A^-|$ onto modes with negative eigenvalue) and the incoming data for that face:

$$\dot{U}_{N-1} \ \ -= \ \tau \cdot \frac{(H^{-1})_{N-1,N-1}}{h} \cdot |A^-| \cdot (U_{N-1} - g). \tag{18}$$

In the $(4, 2)$ SBP norm, $(H^{-1})_{N-1,N-1} = (H^{-1})_{00} = 48/17$ by symmetry.

3.4 Application to Maxwell's equations

For Maxwell's equations extended with constraint damping, the z-axis flux matrix decomposes into four independent 2×2 blocks plus one trivial 1×1 block:

Block	Variables	Eigenvalues	Incoming char. at lower z (w^+)
EM-1	(B_x, D_y)	$\pm c$	$B_x - PP \cdot D_y$
EM-2	(B_y, D_x)	$\pm c$	$B_y + PP \cdot D_x$
Div-B	(B_z, Ψ_B)	± 1	$B_z - \Psi_B$
Div-D	(D_z, Ψ_D)	± 1	$D_z - \Psi_D$
Charge	ρ	0	(none)

Here $c = \sqrt{\varepsilon\mu}^{-1}$ is the local light speed (in our normalization $c = \sqrt{\texttt{ieps} \cdot \texttt{imu}}$) and $PP = \sqrt{\varepsilon/\mu}^{-1} = \sqrt{\texttt{ieps}/\texttt{imu}}$ is the impedance ratio. The divergence blocks have eigenvalues ± 1 because constraint violations propagate at the normalized speed set by the extended system (independent of material). The w^+ column lists, for each block, the left-eigenvector combination that is the incoming characteristic at the lower-z face; the upper-z face injects the opposite combination (w^-) with the signs of the cross-coupling terms reversed.

For the EM-1 block (B_x, D_y), the flux matrix in the convention used by the code is

$$A_z^{(\mathrm{EM1})} = \begin{pmatrix} 0 & -\texttt{ieps} \\ -\texttt{imu} & 0 \end{pmatrix}. \tag{19}$$

The right eigenvector for $\lambda = +c$ is $v^+ = (1, -1/PP)^\top$; the left eigenvector (satisfying $L^+ A = cL^+$) is $L^+ = (1, -PP)^\top$; and the positive projection is

$$A^+ = \frac{c}{L^+ \cdot v^+} v^+ (L^+)^\top = \frac{c}{2} \begin{pmatrix} 1 & -PP \\ -1/PP & 1 \end{pmatrix}. \tag{20}$$

Applying to $(\delta B_x, \delta D_y) = (B_x - g_{B_x}, D_y - g_{D_y})$:

$$(A^+ \delta U)_{B_x} = \tfrac{c}{2}(\delta B_x - PP \cdot \delta D_y), \tag{21}$$

$$(A^+ \delta U)_{D_y} = \tfrac{c}{2}(-\delta B_x/PP + \delta D_y). \tag{22}$$

The incoming characteristic variable for this block is $w^+ = L^+ \cdot U = B_x - PP \cdot D_y$.

Verification on a known solution. For a y-polarised plane wave travelling in $+z$ (incoming at the lower-z boundary), the fields satisfy $D_y = \psi$, $B_x = -\psi$ (the sign makes $\mathbf{E} \times \mathbf{B}$ point in $+\hat{z}$), so $w^+ = -\psi - PP \cdot \psi = -(1 + PP)\psi \neq 0$ as expected for an incoming mode — nonzero, so the SAT penalty actually injects. A sign error that flipped any coupling term (e.g. $+PP$ instead of $-PP$) would give $w^+ = 0$ for this same solution, and the SAT would silently fail to inject the wave. Analogous derivations for the other three blocks give the full SAT formula for the lower-z face; the formulas for the remaining five faces follow by cyclic permutation of axes and sign reversal of the coupling terms for the upper faces (Appendix A).

The implementation lives in `maxwell_eqs.c` as six preprocessor macros: `APPLY_SAT_LOWER_{X,Y,Z}` and `APPLY_SAT_UPPER_{X,Y,Z}`. Only the lower-z face has nonzero g (the incoming source field, Sec. 4); the other five faces use $g = 0$, which corresponds to a perfectly absorbing boundary for waves leaving the domain.

3.5 Edges and corners

At an edge (where two physical faces meet) and at a corner (three faces), the SAT penalties are *additive*: each face contributes its own penalty term independently. No cross-term or corner-specific correction is needed. This additivity is a direct consequence of the tensor-product structure: the 3D discrete energy norm $H = H_x \otimes H_y \otimes H_z$ decomposes the boundary contributions into independent surface integrals over each face, each cancelled by its own SAT term.

Concretely, at the corner $(i, j, k) = (0, 0, 0)$ with all three axes physical, the RHS receives

$$\dot{U}_{0,0,0} \mathrel{-}= \tau \left[\frac{H_0^{-1}}{dx} A_x^+ \delta U + \frac{H_0^{-1}}{dy} A_y^+ \delta U + \frac{H_0^{-1}}{dz} A_z^+ \delta U \right], \tag{23}$$

where the three projections act on the *same* state δU but with axis-specific flux matrices. This is the *only* corner-aware term in the code — the SBP derivatives at the corner use one-sided closures in each direction independently, and no other correction is applied.

3.6 Stability, τ, and CFL at corners

The standard SBP-SAT stability theorem (Svärd & Nordström) requires $\tau \geq 1/2$ for a *single-face* penalty on a scalar hyperbolic equation. For a system with n characteristic speeds and a tensor-product multi-dimensional grid, the same bound applies per face. In the code we use $\tau = 1$ as a comfortable margin.

At corners, three SAT terms with per-face scaling $\tau(H_0^{-1})/h$ stack additively on the same boundary row. The effective spectral radius of the discrete evolution operator grows at the corners, and the usable CFL number drops. Empirically, the scheme with all six faces physical is stable at cfl ≈ 0.2 with classical RK4 — tighter than the cfl ≈ 0.5 safe for an interior-only scheme. Raising τ beyond 1 tightens the CFL further; reducing τ toward 1/2 relaxes it at the cost of stability margin.

4 Source models

The lower-z face (or, for the convergence-test source, every physical face) has nonzero boundary data g corresponding to a specified incoming electromagnetic field. Three sources are implemented:

- `plane_wave`: a bump-enveloped plane wave travelling in $+\hat{z}$ (Sec. 4.1). Pair with periodic x, y for the formal convergence reference (Option B).

- `gaussian_beam`: a paraxial collimated beam injected at the lower-z face (Sec. 4.2). Used with all faces non-periodic for the headline beam-through-lens physics demonstration (Option A).
- `te_waveguide_mode`: an exact vacuum Maxwell TE mode of a rectangular unit-cube waveguide, used as SBP-SAT boundary data on every physical face for the four-configuration convergence study (Sec. 8).

Selection is via the TOML key `type` under the `[source]` table. Per-source parameters live in dedicated sub-tables `[source.plane_wave]`, `[source.gaussian_beam]`, `[source.te_waveguide_mode]`: the driver parses all three (so switching sources is a one-line `type` edit), but only the sub-table matching the active `type` is read by the analytic source functions.

4.1 Plane wave source

The plane-wave source is a transverse electromagnetic wave travelling in $+\hat{z}$, linearly polarised as $\mathbf{E} = (E_0^x\hat{x} + E_0^y\hat{y})f(t-z)\cos(k(z-t))$, modulated by a compactly supported smooth envelope f that turns on over the interval $[a, b]$. In the code this is `incoming_plane_wave`; the envelope is a C^2 polynomial bump. The wave satisfies the vacuum Maxwell equations exactly, independent of wavenumber k, so in a configuration with x, y periodic (and z non-periodic) this source provides an *exact* reference solution for formal convergence testing of the SBP-SAT scheme (Sec. 7).

The plane-wave source has no transverse (x, y) structure — the field at $x = 0$ and $x = L_x$ is identical. Combining this source with non-periodic x, y boundaries therefore produces spurious reflections: the full-amplitude wavefield at the transverse walls is pulled toward zero by the $g = 0$ SAT there, exciting high-frequency numerical modes and ultimately destabilizing the simulation. The plane wave is not usable with six-face SAT.

4.2 Paraxial Gaussian beam

The Gaussian-beam source models a collimated laser beam entering at the lower-z face, optionally converging toward a waist at $z = z_w$. In the paraxial approximation, the scalar amplitude is

$$\psi(\mathbf{x}, t) = E_0 \cdot \frac{w_0}{w(z)} \exp\left(-\frac{r^2}{w(z)^2}\right) \cos\left(kz - \omega t + \frac{kr^2}{2R(z)} - \varphi_G(z)\right), \tag{24}$$

where $r^2 = x^2 + y^2$, $\omega = kc$ (with $c = 1$ in vacuum units), and

$$z_R = \tfrac{1}{2}kw_0^2 \qquad \text{(Rayleigh range)} \tag{25}$$

$$w(z) = w_0\sqrt{1 + ((z - z_w)/z_R)^2} \tag{26}$$

$$\frac{1}{R(z)} = \frac{z - z_w}{(z - z_w)^2 + z_R^2} \tag{27}$$

$$\varphi_G(z) = \arctan((z - z_w)/z_R) \qquad \text{(Gouy phase).} \tag{28}$$

The field is taken linearly polarised along $\hat{x}$ to leading paraxial order: $D_x = \psi$, $B_y = \psi$, all other components zero.

Convergence / divergence. The sign of $1/R(z)$ determines whether the wavefront is converging or diverging. For $z < z_w$, $R < 0$: the wavefronts are concave toward the beam axis, the beam *converges* toward the waist. For $z > z_w$, $R > 0$: the beam diverges. The beam width at the entrance face $z = 0$ is

$$w(0) = w_0\sqrt{1 + (z_w/z_R)^2}, \tag{29}$$

which equals w_0 when $z_w = 0$ (beam at its natural focus at the entrance) and grows with z_w. Placing z_w in the middle of the domain produces a beam that converges through the first half, reaches minimum spot at the middle, then diverges.

Collimation. A beam is "nearly collimated" (laser-like) over a propagation distance L when $z_R \gg L$. The spot size varies by

$$\frac{w(L)}{w_0} = \sqrt{1 + (L/z_R)^2}, \tag{30}$$

so $z_R = 10L$ gives 0.5% broadening, and $z_R = 3L$ gives 5%. For fixed domain L this requires large kw_0^2, which costs transverse resolution: see Sec. 5.

Paraxial residual. Expression (24) satisfies the full Maxwell equations only to $O(1/(kw_0)^2)$. The residual manifests as small but nonzero $\nabla \cdot \mathbf{E}$ and $\nabla \cdot \mathbf{B}$ when the beam is injected. The constraint-damping fields Ψ_D and Ψ_B absorb and radiate away this residual on timescale $1/\kappa_{D,B}$. A sufficiently paraxial beam ($kw_0 \gtrsim 5$) leaves a residual below one part in 25, well within the constraint- damping system's domain of validity.

4.3 Smooth turn-on in time

At $t = 0$, the full-domain initial data is filled with the analytic source evaluated at $t = 0$. For the plane-wave source, the envelope $f(t - z)$ vanishes at $t = 0$ for all $z \geq 0$, so the initial state is identically zero and the wave enters gradually through the SAT boundary.

For the Gaussian beam, the analytic formula (24) does not vanish at $t = 0$: filling the interior with it would seed the domain with a state that is only a paraxial approximation to a discrete solution. Empirically, this excites a numerical instability that grows exponentially and overwhelms the simulation on a timescale of a few light-crossing times, regardless of CFL. To avoid this, the beam source is multiplied by a smooth C^2 turn-on envelope $f(t, t_a, t_b)$ that rises from 0 at $t \leq t_a$ to 1 at $t \geq t_b$. The turn-on interval $[t_a, t_b]$ is exposed as `ramp_a` and `ramp_b` under `[source.gaussian_beam]`; the default is $[0, 1]$. The initial state is therefore identically zero, and the beam grows in through the boundary SAT as the envelope rises. After $t \geq t_b$ the source is at full amplitude and the simulation runs in its intended regime. Lengthening the ramp (e.g. $t_b = 3$) reduces the paraxial-residual excitation during turn-on, at the cost of delaying the onset of interesting physics; this is useful when investigating the residual at high $k \cdot w_0$.

The `te_waveguide_mode` source is an exact solution of the continuum Maxwell PDE and therefore needs no turn-on — it is non-zero at $t = 0$, and the initial state on the full grid is consistent with the boundary data at every face. The convergence- test routine relies on this consistency (Sec. 8).

5 Limitations

5.1 Transverse-face reflections

The $g = 0$ SAT on the $\pm x$ and $\pm y$ faces is correct only insofar as the physical solution has negligible amplitude at those walls. For a Gaussian beam with $w_0 \ll L_x, L_y$ the tail at the wall decays as $\exp(-(L_{x,y}/2w)^2)$ and reflections are below roundoff for $L_{x,y} \gtrsim 10w_0$. For a plane wave (which has full amplitude at the walls) this boundary treatment is not appropriate — the spurious reflection is large and cascades into an instability.

5.2 Paraxial approximation

The Gaussian-beam source is a paraxial approximation to Maxwell's equations. The residual constraint violation is $O(1/(kw_0)^2)$ and is absorbed by the constraint-damping fields. This is fine as a physics source — real laser beams are described by the same paraxial formula — but means the beam is not an exact solution of the full Maxwell system. The L^2 error of the numerical evolution against the paraxial reference is therefore not a convergence metric for the scheme; use the plane-wave source for that.

5.3 Resolution / containment / paraxial tradeoff

Three constraints on the beam parameters:

- **Grid resolution:** $k \cdot dx \lesssim 0.6$, i.e. roughly 10 points per wavelength.
- **Paraxial validity:** $kw_0 \gtrsim 5$, giving residual $\leq 4\%$.
- **Transverse containment:** $L_x/w_0 \gtrsim 5$ (stronger: ≥ 10) so the Gaussian tail is negligible at the x, y walls.

For a laser-like (nearly-collimated) beam, the Rayleigh range $z_R = kw_0^2/2$ must additionally be large compared with the propagation length L_z. All four constraints together favour large k and large w_0, which raises the required N_x steeply. Practical configurations balance collimation against computational cost.

5.4 CFL at corners

With all six faces treated by SAT, the scheme is stable at cfl ≈ 0.2 with classical RK4 — tighter than the cfl ≈ 0.5 safe for the periodic-xy interior-only scheme. This is a consequence of the three SAT penalties stacking at corners (Sec. 3). If you see exponentially-growing L^2 error, reduce the CFL before suspecting the physics.

5.5 Plane wave with six-face SAT

The plane-wave source combined with all six faces set to SAT produces a numerical instability (the transverse walls see full-amplitude tangential field and the $g = 0$ SAT attempts to force it to zero). The plane-wave source is supported only with x, y periodic and z non-periodic. Attempting the six-face-SAT plane-wave run is *not* a bug; it is the documented failure mode of the SAT boundary for a physical configuration whose field is not well-contained in the transverse plane.

5.6 Angular (oblique) reflections from every SAT face

The most important limitation of the current boundary treatment is one that all characteristic-based absorbing boundaries share: the SAT penalty is exact for waves moving *normal* to the boundary, but only partial for waves moving at oblique angles. This affects every face, not just the transverse ones:

- **Lower-z face**: the SAT's $+c$ projection correctly leaves a head-on outgoing wave ($-c$ mode) untouched. A wave moving at angle θ to the z-axis has its energy split between the $+c$ and $-c$ z-characteristics in proportions that depend on $\sin\theta$. The SAT treats the $+c$ part as incoming and pulls it toward the prescribed beam data, producing a spurious reflection whose amplitude grows with θ.

- **Upper-z face**: same mechanism, with $g = 0$. A wave refracted by the lens and travelling mostly in $+z$ but at an angle is partially reflected back into the domain.
- **$\pm x$, $\pm y$ faces**: same mechanism yet again. The beam reaches these walls only as a Gaussian tail and as lens-scattered components, so their contribution is small for well-contained beams but non-negligible whenever the lens refracts energy sideways.

Concrete failure mode: beam on a high-ε lens. A dielectric step of permittivity $\varepsilon_{\max}$ reflects a Fresnel fraction $R = \left[(\sqrt{\varepsilon_{\max}} - 1)/(\sqrt{\varepsilon_{\max}} + 1)\right]^2$ of incident intensity at normal incidence. For $\varepsilon_{\max} = 4$, $R \approx 0.11$ in intensity, or $1/3$ in amplitude — a substantial portion of the incident beam bounces off the lens and propagates back toward $-z$. The pure-$-z$ component exits cleanly through the lower-z SAT; components scattered at angles by the elliptical side-lobes of the lens are partially reflected and start a second pass through the lens. After a few such bounces the domain fills with low-amplitude oblique modes.

Observable symptoms. High-frequency interference patterns (the incident + reflected beam forming standing-wave nodes every $\lambda/2 = \pi/k$) and visible oblique wavefronts near the lower-z face at late times. The interference is real physics; the oblique wavefronts arriving at the lower-z face are the spurious angular reflections. Both coexist in any long-time run with a moderately strong lens.

Mitigations within the current code.

- **Shorten the simulation time** so no wave has time to complete a round trip.
- **Weaken the lens** ($\varepsilon_{\max}$ closer to 1) so the reflection coefficient is smaller and the artifacts drop proportionally.
- **Widen the domain** so the transverse walls are far from the beam even after the lens refracts it.
- **Enable Kreiss-Oliger dissipation** (`use_dissipation = true`, `diss_coeff` ~ 0.02–0.1) to damp grid-scale noise from partial reflections. This does not remove the angular reflections themselves but suppresses their high-frequency signature.
- **Do not use the current code for long-time resonance studies**: the SAT is not designed to model a cavity with clean walls.

None of these mitigations is a true fix. To absorb oblique waves at all angles, a different class of boundary treatment is required — the Perfectly Matched Layer (next section).

6 Perfectly Matched Layers: a future extension

A Perfectly Matched Layer (PML) is a finite-thickness absorbing buffer that surrounds the physical domain and is designed to absorb outgoing waves *at any angle and any frequency*, with zero reflection at the interface with the physical domain at the continuum level. PML is the standard tool in production electromagnetic and elastodynamic codes for open-domain simulations. The current `Maxwell_Penalty` code does not include a PML — the SBP-SAT boundary is the only absorbing mechanism — and this section is a forward-looking note on what a PML implementation would add and how it would fit.

6.1 The physical picture

The idea, due to Bérenger (1994), is to surround the computational domain with a thin layer (typically 8–20 cells) in which the governing equations are modified so that:

1. A wave crossing the physical-domain / PML interface sees no discontinuity: its wavenumber, direction, and phase velocity are unchanged. No reflection is generated at the interface — this is the "perfectly matched" property.

2. Once inside the layer, the wave decays exponentially as it propagates away from the physical domain.

3. Any residual wave that reaches the outer face of the PML and reflects (from whatever boundary condition is imposed there — a trivial zero Dirichlet suffices) is further attenuated on the return trip through the layer.

The round-trip attenuation through the PML controls the effective reflection the physical domain sees. With a $\sim$ 10-cell layer and a standard σ profile, round-trip amplitude ratios of 10^{-4} to 10^{-6} are routine — orders of magnitude better than the characteristic-based SAT for oblique incidence.

6.2 The mathematical mechanism: complex coordinate stretching

For Maxwell's equations in, say, an x-face PML, one formally replaces the x-derivatives by stretched derivatives

$$\frac{\partial}{\partial x} \longrightarrow \frac{1}{s_x(x)}\frac{\partial}{\partial x}, \qquad s_x(x) \;=\; 1 + \frac{\sigma_x(x)}{i\omega}. \tag{31}$$

The stretch factor s_x is complex; $\sigma_x(x) \geq 0$ is the "PML damping profile", equal to zero in the physical domain and ramping up through the PML region.

Substituting a plane wave $e^{i(kx-\omega t)}$ into the stretched equation gives, in the PML,

$$e^{i(kx-\omega t)} \cdot \exp\left(-\int_{x_0}^{x} \frac{\sigma_x(x')}{c}\, dx'\right),$$

i.e. the same wave with an exponential envelope. Crucially, the wavenumber k and the phase velocity are unchanged, so the wave does not refract at the PML interface — which is why the match is perfect for *all* angles and *all* frequencies.

Similar stretch factors apply at the y and z faces. In a corner region where two or three PMLs overlap, all the corresponding derivatives are stretched; the continuum analysis still gives no corner reflection.

6.3 From frequency domain to time domain

The complex factor $1/s_x = 1/(1 + \sigma_x/(i\omega))$ is trivial in the frequency domain but requires care in the time domain — dividing by $(i\omega + \sigma_x)$ is a convolution in time with an exponential kernel. Two standard implementations:

- **Bérenger split-field PML (1994).** Each field component is split into pieces labelled by the derivative that contributes to it; each piece gets its own damping equation. Simple but doubles the number of field variables in the PML region.

- **Convolutional PML / CPML (Roden & Gedney 2000).** Uses the same complex stretch but implements the convolution directly through one auxiliary field per stretched derivative, evolved by a first-order ODE. Far fewer extra fields, more robust at grazing incidence, and handles inhomogeneous media (like our lens) more gracefully. CPML is the modern standard and would be the recommended choice here.

In CPML, each stretched derivative $(1/s_x)\partial_x E$ in the time-domain PDE is replaced by $\partial_x E + \Psi^x$, where Ψ^x is an auxiliary field evolved by

$$\frac{d\Psi^x}{dt} \;=\; -\alpha_x \Psi^x \;-\; \sigma_x \, \partial_x E,$$

with α_x an optional additional parameter (often zero) that improves performance at grazing incidence. Outside the PML, $\sigma_x = \alpha_x = 0$ and Ψ^x is identically zero; inside, Ψ^x tracks the history of the spatial derivative damped exponentially.

6.4 The damping profile

$\sigma(x)$ must start at zero at the PML / physical-domain interface and rise smoothly to a maximum at the outer face:

$$\sigma(x) \;=\; \sigma_{\max} \left(\frac{x - x_{\text{interface}}}{L_{\text{PML}}} \right)^m, \qquad m = 2, 3, 4 \text{ typical}, \tag{32}$$

with $m = 3$ (cubic profile) a standard choice. The maximum $\sigma_{\max}$ is tuned so the theoretical round-trip reflection

$$R \;=\; \exp\!\left(-\frac{2\sigma_{\max} L_{\text{PML}}}{(m+1)\, c} \right) \tag{33}$$

sits at the target level (typically $R \sim 10^{-5}$). Starting σ exactly at zero at the interface is essential: any discontinuity in σ at that boundary creates a small but visible reflection, and the whole point of a PML is to avoid it.

6.5 What a PML would add to `Maxwell_Penalty`

Adding CPML to the six faces would require, roughly:

1. **Grid extension.** Physical domain padded by $N_{\text{PML}} \approx 10$ cells on each physical face, so $N_x \to N_x + 2N_{\text{PML}}$ etc. A moderate overhead; for $N = 64$, a 10-cell PML per face grows the grid to 84 per axis, so the total work grows by $(84/64)^3 \approx 2.3\times$.

2. **Auxiliary fields.** In CPML one needs six $\Psi^{\text{axis}}_{\text{component}}$ fields (one per stretched derivative per field component), evolved on the PML region only. Memory overhead is modest because Ψ is zero outside the PML; storing it only in the layer keeps the overhead to $\sim 30\%$ of the PML-region cost.

3. **RHS extension.** The `SIMPLE_MAXWELL_INTERIOR_DOT` macro would need a PML-aware cousin that includes the Ψ contributions and evolves them. Because σ varies with position, the PML body is not easily macro-ised with fixed coefficients; a small per-point helper function is more natural.

4. **Constraint-damping coverage.** The extended Maxwell system here carries Ψ_D and Ψ_B with their own constraint-propagation speeds (normalized to 1, not c). These blocks need their own PML damping profile, usually tuned to match the same round-trip attenuation target. The constraint-damping and EM PMLs have different physical wavelengths so their $\sigma_{\max}$ values differ.
5. **Outer boundary.** The outermost cell of the PML can use a trivial zero-Dirichlet boundary. A zero SAT would also work. The round-trip attenuation makes the choice effectively irrelevant.
6. **MPI topology.** The PML adds layer cells on ranks that own physical faces, but the MPI decomposition is otherwise unchanged. Ghost-zone synchronisation on the interior MPI faces works as before.

The complete implementation effort is on the order of a few hundred lines of code and a week of careful convergence testing. The payoff is clean simulations of open-domain scattering problems — a lens demo, a beam refraction study, a waveguide mode leakage analysis — with artifacts driven below the single-precision noise floor.

6.6 Summary: SAT versus PML

	SBP-SAT (current)	PML (future extension)
Mechanism	Characteristic projection	Complex coordinate stretching
Normal incidence	Exact (for $\tau \geq 1/2$)	Near-exact, $R \sim 10^{-5}$
Grazing incidence	Strongly reflective	Near-exact
Extra grid cells	None	~ 10 per physical face
Extra fields	None	One Ψ per stretched derivative
Energy estimate	Yes (provable)	Not directly; effective via round-trip bound
Handles lens refl.	Head-on only	All angles
Code complexity	Low	Medium–high

The choice between SAT and PML is a design decision, not a correctness one: both can be implemented correctly; they just handle different regimes well. For a teaching code demonstrating SBP-SAT, SAT alone is appropriate. For production scattering simulations, PML is the right tool.

7 Using the code

7.1 Build

Dependencies: MPI (tested with OpenMPI), HDF5 (with the high-level interface), a C11 compiler, and a C++17 compiler (for the TOML parameter parser).

From `src/`:

```
cmake -S . -B build
cmake --build build -j
```

or equivalently with the hand-written Makefile:

```
make maxwell_system
```

The resulting executable is `maxwell_system`. A test suite (domain decomposition, ghost-zone synchronization, RK4 convergence) is run with

```
ctest --test-dir build
```

7.2 Running

```
mpirun -np <N> ./maxwell_system maxwell.toml
```

Output is written to `3D_rank_<R>.h5` for each rank (one file per MPI rank, one group per output step). See Sec. 7.6 for how to visualize these files.

7.3 Parameter file structure

The TOML parameter file has the following sections:

[grid] Domain extent and resolution.

```
nx = 64 # global grid points in x (total, not per rank)
ny = 64
nz = 128
x0 = -1.0 # lower corner
y0 = -1.0
z0 = 0.0
xn = 1.0 # upper corner
yn = 1.0
zn = 4.0
periodic_x = false # if true, no SAT on this axis, ghost zones wrap
periodic_y = false
periodic_z = false
```

[solver] Numerical parameters.

```
ghost_size = 3
cfl_factor = 0.2 # use <= 0.2 for all-physical; <= 0.5 for periodic x,y
max_iterations = 2049
output_every = 32
checkpoint_every = 512
max_checkpoints = 2
recover = false
use_dissipation = false # Kreiss-Oliger, off by default
diss_coeff = 0.1
tau = 1.0 # SAT penalty strength, >= 1/2
```

[physics] Constraint-damping rates.

```
kappa_D = 0.1
kappa_B = 0.1
```

[source] Selects the incoming source type. Per-source parameters live in dedicated sub-tables, of which exactly one is consumed per run (matching the `type` value); the others are parsed but ignored, so switching sources is a one-line `type` edit with no other TOML shuffling.

```
[source]
type = "gaussian_beam" # or "plane_wave" or "te_waveguide_mode"
```

[source.plane_wave] Plane-wave parameters. The envelope turn-on interval $[a, b]$ is in $t - z$ (characteristic) coordinate.

```
[source.plane_wave]
ax = 1.0 # x-polarisation amplitude
ay = 1.0 # y-polarisation amplitude
k = 4.0 # wavenumber
bump_a = 0.0 # envelope turn-on start
bump_b = 2.0 # envelope turn-on end
```

[source.gaussian_beam] Paraxial Gaussian-beam parameters. The temporal ramp `[ramp_a, ramp_b]` is the $[t_a, t_b]$ interval of the smooth C^2 turn-on envelope (Sec. 4.2); lengthen it to reduce the paraxial- residual excitation during turn-on.

```
[source.gaussian_beam]
w0 = 0.20 # waist radius
z_waist = 2.0 # z location of waist (inside domain for lens demo)
k = 15.0 # wavenumber
amplitude = 1.0 # peak field amplitude
ramp_a = 0.0 # turn-on start
ramp_b = 1.0 # turn-on end
```

[source.te_waveguide_mode] Rectangular waveguide TE mode numbers. `l, m` are the transverse standing-wave orders in x, y; `n` is the longitudinal (propagation-direction) order in z. All three must be ≥ 1. Used for the convergence study (Sec. 8).

```
[source.te_waveguide_mode]
l = 2
m = 2
n = 2
```

[material] The selector `epsilon_type` chooses how permittivity varies in space. `[material.background]` carries the always-used uniform scalars (permittivity, permeability, conductivity); when `epsilon_type = "elliptical"`, `[material.elliptical]` additionally describes a lens-shaped bump layered on top of the background via

$$\varepsilon(x, z) = (\varepsilon_{\max} - \varepsilon_{\text{bg}}) \exp\left[-s^2 \left(\frac{(x - x_0)^2}{a^2} + \frac{(z - z_0)^2}{b^2}\right)^2\right] + \varepsilon_{\text{bg}},$$

where ε_{bg} is the `epsilon` value from `[material.background]`. With the default $\varepsilon_{\text{bg}} = 1$ this reduces to the vacuum-background lens used for the headline physics demo.

```
[material]
epsilon_type = "elliptical" # or "uniform"

[material.background]
epsilon = 1.0 # baseline permittivity (asymptote of the elliptical profile)
mu = 1.0
sigma = 0.0

[material.elliptical]
max = 4.0 # peak permittivity at the lens centre
x0 = 0.0 # centre x
```

```
z0 = 2.0 # centre z
s = 2.0 # steepness
a = 0.5 # semi-axis x
b = 0.25 # semi-axis z
```

7.4 Two canonical configurations

Option B: plane-wave convergence reference. Periodic x, y, non-periodic z, plane-wave source. The only configuration in which an exact, non-dispersive travelling-wave solution exists for the plane-wave source (non-periodic transverse faces would force a $g = 0$ SAT at full-amplitude wavefield points and destabilise the run; see Sec. 5):

```
[grid]
periodic_x = true
periodic_y = true
periodic_z = false

[source]
type = "plane_wave"
```

On a sequence of refined grids, the L^2 error relative to the analytic solution reduces at the order of the SBP-$(4,2)$ boundary closure (formally second-order locally at the boundary, integrating to higher order globally; empirically around third-order).

Option A: beam through lens. All six faces non-periodic with SAT, Gaussian-beam source, elliptical permittivity. This is the "headline" physics demonstration: a collimated (or mildly convergent) laser beam enters at $z = 0$, propagates through a lens-shaped high- permittivity region at $z \approx z_w$, and exits at $z = L_z$.

```
[grid]
periodic_x = false
periodic_y = false
periodic_z = false

[source]
type = "gaussian_beam"

[material]
epsilon_type = "elliptical"
```

7.5 Output layout

Each rank writes a self-describing HDF5 file `3D_rank_<R>.h5` containing the full field state at every output interval, grouped by output index. Groups $/0, /1, /2, \ldots$ correspond to the initial state, the first output step, and so on (step N is at simulation time $N \cdot$ `output_every` $\cdot\, dt$). Each group has datasets for the nine evolved variables and the five auxiliary variables (including the diagnostic divergences c_D, c_B). The `/metadata` group carries per-rank attributes describing the grid (origin, spacing, local and global sizes, ghost width, variable names) — everything needed to reassemble the global domain without any external bookkeeping.

Note that with the beam source's smooth turn-on, group /0 contains identically zero fields. The first non-trivial output is at $t =$ `output_every` $\cdot\, dt$; by $t \geq$ `ramp_b` the turn-on envelope has saturated and the beam is at full amplitude.

An L^2 diagnostic `l2_norm.dat` is written by rank 0 every output step, containing $(t, \|U_{\text{numerical}} - U_{\text{analytic}}\|_{L^2})$. For the plane-wave source with periodic x, y this is a convergence metric; for the Gaussian-beam source the analytic reference is only the paraxial approximation and the L^2 error reaches a paraxial-scale plateau, not a convergence-to-zero.

7.6 Visualization

The raw HDF5 layout is not directly readable by ParaView, VisIt, or PyVista — those tools expect either a VTK-native format or a sidecar manifest that describes the grid. The preferred workflow is to emit an XDMF sidecar after the run:

```
python scripts/make_xdmf.py --dir <run_dir> --dt-per-output <T>
```

where `<T>` is the physical time between successive HDF5 output groups, i.e. $T =$ `output_every` $\cdot$ `cfl_factor` $\cdot \min(dx, dy, dz)$. The script scans the per-rank HDF5 files, reads their `/metadata`, and emits a single `maxwell.xmf` manifest alongside the HDF5 files. The manifest is pure XML metadata — the payload bytes stay in HDF5 and are never copied.

The resulting `.xmf` file is a direct input for all three target tools:

- **ParaView**: `File` → `Open` → `maxwell.xmf`. The Xdmf3 reader is preferred; the legacy Xdmf2 reader also works.
- **VisIt**: `File` → `Open` → `maxwell.xmf`. VisIt's Xdmf plugin reads it directly.
- **PyVista**: `pyvista.XdmfReader("maxwell.xmf")`.

A time slider stepping through the N output steps appears automatically; all 14 fields are selectable as point-data attributes.

Grid-layout details. The XDMF uses a 3DRectMesh topology with an explicit VXVYVZ geometry (one per-axis coordinate list). This avoids the long-standing XDMF ORIGIN_DXDYDZ ambiguity whereby VisIt and ParaView disagree on whether the origin tuple is listed in spatial (x, y, z) order or in topology-dimensions (z, y, x) order: VXVYVZ tags each coordinate list explicitly with its axis role, so both readers agree.

Each MPI rank appears as a spatial sub-grid inside each time step. Adjacent ranks share one grid plane at each internal MPI boundary (one extra ghost row is included on the upper side of every MPI boundary), so the rendered cells tile the global grid without gaps. The shared points are filled from the same ghost-exchanged data before output, so the two neighbours render identical values at the shared plane and the overlap is invisible.

Legacy path. The older `scripts/hdf5_to_vtk.py` reassembles the global grid on rank 0 and writes one VTI per time step. It is still supplied as a fallback for environments where XDMF is not convenient, but it is serial, duplicates the data on disk, and has historically surfaced platform-specific portability bugs (e.g., a NumPy 2.0-era issue in which `str(np.float64(x))` became `"np.float64(x)"` and leaked into the VTI XML header; if you have pre-existing VTI files with that issue, a tiny in-place patcher `scripts/fix_vti_header.py` corrects the header without rewriting the data payload).

7.7 Diagnostics and common problems

L^2 error blows up exponentially. Usually CFL is too large. Reduce `cfl_factor` to 0.2 (six-face SAT) or verify the configuration is in fact Option B if you expected cfl = 0.4 to work.

D field looks "empty" in visualization. Check the time slice. Group /0 is the initial state, which is zero for both the plane-wave and Gaussian-beam sources (the turn-on envelope). For the default ramp $[0, 1]$ and the default `output_every` on the shipped TOML, the beam has turned on to $\sim 3\%$ by group /1 ($t = 0.2$) and reaches full amplitude by $t \geq$ `ramp_b`; visualize group /5 or later for the default parameters. (The `te_waveguide_mode` source has no ramp and its group /0 is already the full analytic field.)

Paraxial residual looks larger than expected. Verify $k \cdot w_0 \gtrsim 5$. Residual scales as $1/(kw_0)^2$.

Beam appears to "reflect" off the transverse walls. The x, y faces use $g = 0$ SAT; the reflection amplitude is proportional to the field amplitude at the wall. Increase L_x, L_y, or reduce w_0 so the tail at the wall is negligible. Rule of thumb: $L_x \geq 10w_0$ where w_0 here should be the local spot size along the propagation, not just the waist.

At late times, high-frequency structure and oblique wavefronts appear near the lower-z face. This is the combined signature of a real standing wave (incident + lens-reflected beam interfering every $\lambda/2 = \pi/k$) and spurious angular reflections of lens-scattered energy off every SAT face. The first is physics; the second is a known limitation of the SAT boundary treatment — see Sec. 5.6. Enabling Kreiss-Oliger dissipation (`use_dissipation = true`, `diss_coeff` ~ 0.05) suppresses the high-frequency component. For truly angle-independent absorption the code would need a PML — see Sec. 6.

8 Verification: convergence testing

A direct test for the correctness of the interior stencil, each face's SBP-SAT closure, and the additive penalty at edges and corners is obtained by forcing the numerical solution to converge to a known analytic solution of the continuum PDE and measuring the convergence rate. This section describes the test setup implemented in the code and how to run it.

8.1 Test problem

The test uses the `te_waveguide_mode` function from `analytic_solutions.c`, which evaluates an exact vacuum Maxwell solution on $[0, 1]^3$ with uniform permittivity $\varepsilon = \mu = 1$, zero conductivity, and no sources. The solution is the TE rectangular-waveguide mode with transverse standing-wave numbers (l, m) in (x, y) and longitudinal propagation mode n in z:

$$
\begin{aligned}
D_x(x, y, z, t) &= \frac{\omega}{l\pi} \sin(m\pi y) \cos(l\pi x) \cos(n\pi z - \omega t), \\
D_y(x, y, z, t) &= -\frac{\omega}{m\pi} \sin(l\pi x) \cos(m\pi y) \cos(n\pi z - \omega t), \\
D_z(x, y, z, t) &= 0, \\
B_x(x, y, z, t) &= \frac{n}{m} \sin(l\pi x) \cos(m\pi y) \cos(n\pi z - \omega t), \\
B_y(x, y, z, t) &= \frac{n}{l} \sin(m\pi y) \cos(l\pi x) \cos(n\pi z - \omega t), \\
B_z(x, y, z, t) &= -\frac{l^2 + m^2}{lm} \sin(n\pi z - \omega t) \cos(l\pi x) \cos(m\pi y),
\end{aligned}
$$

with constraint fields $\Psi_D = \Psi_B = \rho = 0$ and the dispersion relation $\omega = \pi\sqrt{l^2 + m^2 + n^2}$. The l, m, n mode numbers are set under `[source.te_waveguide_mode]` in the TOML (all three integer, ≥ 1; the parameter parser enforces the positivity bound, which also rules out the $1/(lm)$ division below). The shipped default is $(l, m, n) = (2, 2, 2)$. The solution satisfies $\nabla \cdot \mathbf{D} = \nabla \cdot \mathbf{B} = 0$ identically; for even (l, m, n) it is also periodic on the unit cube with period 1, so the same reference serves both periodic and fully-physical boundary configurations in the study below. The waveguide mode has nontrivial structure in every direction — perfect for exercising all parts of the scheme.

8.2 Using the analytic as SAT boundary data

When `[source] type = "te_waveguide_mode"`, the code routes the analytic solution as the boundary data g on *every* physical face (not just the lower-z face as in the plane-wave and beam modes). For each SAT-face point, the scheme evaluates the analytic at the local (x, y, z, t) and passes it into the relevant `APPLY_SAT_*` macro. The numerical state U is then driven toward the analytic through the incoming-characteristic projection of every physical face; the outgoing characteristics are untouched, so the test is consistent with the method's usual energy argument.

Concretely, the scheme being tested is:

- Deep interior: 4th-order centered SBP stencil `D4CEN` in all three directions.
- Near-boundary rows on physical axes: SBP 4-2 one-sided closures (rows 0–3 at lower face, rows $N-4 \ldots N-1$ at upper face; see Sec. 2.2).
- Exactly one boundary row per physical face: SAT penalty with g evaluated from the analytic at that point.
- At edges (two physical faces meeting): two SAT penalties fire additively.
- At corners (three faces meeting): three SAT penalties fire additively.
- Time integration: classical explicit RK4.

If any of these pieces is implemented incorrectly, the convergence rate at the affected configuration will fall below the theoretical expectation. The decomposition below localises which class of boundary structure is broken.

8.3 The four configurations

The boundary periodicities are varied per axis to change which kinds of boundary structures are exercised:

Configuration	`periodic_x`	`periodic_y`	`periodic_z`	Physical faces / edges / corners	Expected L^2 rat
`fully_periodic`	true	true	true	0/0/0	4
`z_physical`	true	true	false	2/0/0	3
`yz_physical`	true	false	false	4/4/0	3
`all_physical`	false	false	false	6/12/8	3

What each configuration localises.

- `fully_periodic`: no SBP closures, no SAT penalties. All derivatives are `D4CEN` with periodic ghost zones. Tests only the interior stencil. Expected rate: 4.
- `z_physical`: only the two z-axis faces are physical; x, y stay periodic. Tests the interior plus the two z-face SBP closures plus the SAT penalty at the single lower-z and upper-z boundary rows. No physical edges or corners are present. A drop in the rate here indicates a problem with either the SBP closure or the SAT penalty on a face.
- `yz_physical`: adds four y-z edges (where the physical y and z faces meet). A rate drop moving from `z_physical` to `yz_physical` indicates the additive edge penalty is mis-handled.
- `all_physical`: the full setup — six faces, twelve edges, eight corners. A rate drop at this step would indicate the three-face corner penalty is mis-handled.

8.4 Why the expected rates are 4 and 3

The scheme is formally:

- 4th-order in the interior (the `D4CEN` stencil is $O(h^4)$).
- 2nd-order locally at SBP-4-2 boundary rows (the Strand closures are $O(h^2)$ at each of the first/last four rows).

For a hyperbolic system discretized by an SBP operator with interior order $2p$ and boundary truncation of order s, the classical Svärd-Nordström result predicts a global L^2 convergence rate of $\min(2p, s+1)$ at late times; for SBP-4-2 with $2p = 4$ and $s = 2$ this gives rate 3. The interior-only configuration (periodic everywhere) sidesteps the boundary closures and retains the full 4th-order rate.

Two practical notes on these predictions:

- The $s+1$ rule applies to each physical face. Edges and corners do not further degrade the rate: the additive SAT penalty means each boundary contributes a separate energy estimate, and the rates combine as a maximum of errors, not a product. This is the feature being tested.
- Measured rates often approach 3 from above at finite N — rates of 3.1–3.2 at $N = 48$ are typical for the SBP-4-2 scheme; they asymptote to exactly 3 as $N \to \infty$. A rate persistently *below* 3 at moderate N indicates a real problem.

8.5 Running the convergence test

The driver `scripts/convergence_test.py` generates a TOML file per (configuration, resolution) pair, runs the solver, parses `l2_norm.dat` for the final-time error, and fits the convergence rate between consecutive resolutions.

```
python scripts/convergence_test.py \
    --solver src/build/maxwell_system \
    --resolutions 16 24 32 48 \
    --final-time 0.1 \
    --np 4
```

Options:

- `-solver PATH`: path to the built `maxwell_system` executable.
- `-resolutions N1 N2 ...`: list of per-axis grid sizes to run. Default: `16 24 32 48`. A cubic grid $N \times N \times N$ is used for every run.
- `-final-time T`: physical simulation time per run (default 0.1). Large enough that a few wave periods elapse; small enough that the error is dominated by the discretization rather than accumulated round-off.
- `-cfl C`: CFL factor (default 0.2).
- `-np N`: number of MPI ranks (default 2; 4 is recommended for the $N = 48$ run to keep wall-clock under a minute).
- `-work-dir DIR`: scratch directory for per-run TOMLs and HDF5 outputs.

The driver runs the four configurations in sequence. Each configuration's section prints the per-resolution errors and the pairwise rates:

```
=== all_physical  (expected L2 rate 3.0) ===
  N= 16 ... L2 = 0.01092
  N= 24 ... L2 = 0.003261
  N= 32 ... L2 = 0.001328
  N= 48 ... L2 = 0.000368
  resolution                16         24         32         48
  L2 error             0.01092   0.003261   0.001328   0.000368
  rate N_k,N_k+1          None      2.981      3.122      3.166
```

A summary table compares the measured final rates against their expected values:

```
=== SUMMARY ===
config              N=16        N=24        N=32        N=48  final rate
fully_periodic  0.001041   0.0002078   6.597e-05   1.306e-05    3.994   OK
z_physical       0.00739    0.002273   0.0009391    0.000261    3.158   OK
yz_physical     0.009489    0.002867    0.001174    0.000325    3.167   OK
all_physical     0.01092    0.003261    0.001328    0.000368    3.166   OK
```

A rate is flagged as `SUSPICIOUS` if it deviates from the expected value by more than 1.0, and the driver exits nonzero. The current code has all four rates at their expected values.

8.6 Baseline results (as of the validated build)

The numbers above are the validated baseline on the code in its current state. For future regression checks:

Configuration	$N = 16$	$N = 24$	$N = 32$	$N = 48$	Final rate
`fully_periodic`	$1.04 \cdot 10^{-3}$	$2.08 \cdot 10^{-4}$	$6.60 \cdot 10^{-5}$	$1.31 \cdot 10^{-5}$	3.99
`z_physical`	$7.39 \cdot 10^{-3}$	$2.27 \cdot 10^{-3}$	$9.39 \cdot 10^{-4}$	$2.61 \cdot 10^{-4}$	3.16
`yz_physical`	$9.49 \cdot 10^{-3}$	$2.87 \cdot 10^{-3}$	$1.17 \cdot 10^{-3}$	$3.25 \cdot 10^{-4}$	3.17
`all_physical`	$1.09 \cdot 10^{-2}$	$3.26 \cdot 10^{-3}$	$1.33 \cdot 10^{-3}$	$3.68 \cdot 10^{-4}$	3.17

Run parameters: $T_{\text{final}} = 0.1$, cfl $= 0.2$, $\tau = 1.0$, four MPI ranks, uniform $\varepsilon = \mu = 1$, $\sigma = 0$, analytic mode $(l, m, n) = (2, 2, 2)$.

8.7 Interpreting a failed test

If a future change to the boundary code causes one of the rates to drop, the configuration hierarchy localises the fault:

- `fully_periodic` rate drops below 4: the interior `D4CEN` stencil, the ghost-zone sync, or the periodic wrap is broken.
- `z_physical` rate drops below 3 while `fully_periodic` stays at 4: the z-face SBP closures or the lower-/upper-z SAT penalty is broken.
- `yz_physical` rate drops below 3 while `z_physical` is fine: the additive edge penalty at the y-z edges is broken. (Since the y and z face SATs are each known to work in isolation from the previous row, the edge additivity is the new thing being tested.)
- `all_physical` rate drops below 3 while `yz_physical` is fine: the three-face corner penalty at x-y-z corners is broken.

This localisation is exactly the purpose of the test matrix. A successful pass on all four rows confirms that every part of the algorithm — interior stencil, face closure, face SAT, edge additivity, corner additivity — is correctly implemented.

A subtle point: why the oblique-reflection failure mode of Sec. 5.6 does not break this test. The `te_waveguide_mode` has non-trivial structure in every direction, and hits the $\pm x, \pm y$ faces at oblique angles — the very configuration that is mis-handled by the $g = 0$ SAT used in the lens demo. One could reasonably expect spurious reflections from the x and y faces to pollute the measured rate.

This does not happen because the convergence test uses $g =$ analytic on every physical face, not $g = 0$. The absorbing-boundary failure mode only arises when the prescribed incoming data disagrees with the true physical solution at the boundary: with $g = 0$ and a nonzero field tangent to the wall, the SAT pulls the wave toward zero instead of toward its true value, and the resulting mismatch is the spurious reflection. When g equals the exact analytic at every boundary point, the SAT imposes the *correct* incoming data on every face, regardless of the incidence angle. Mathematically, the SAT penalty is a weak-form Dirichlet imposition of the incoming characteristic; with the correct right-hand side it has no angular blind spot.

The convergence test therefore verifies that the SBP-SAT scheme, *given the correct incoming data*, converges to the analytic at the expected rate. It is a test of the scheme's implementation, not of the quality of absorbing boundaries in the lens-demo sense.

Caveats.

- The `te_waveguide_mode` source requires $\varepsilon = \mu = 1$ and $\sigma = 0$ (set in `[material.background]` by the driver). The test does not exercise the spatially-varying permittivity code path; that is a separate physics question, not a convergence question.
- The constraint fields Ψ_D, Ψ_B are identically zero in the analytic. The constraint-damping blocks of the SAT are exercised only by their coupling to the EM blocks through residual numerical constraint violations, so a bug localised entirely to the Ψ_D, Ψ_B blocks might not be caught at full sensitivity by this test. Production deployment with non-vacuum materials or with sources is the right check for those blocks.

- The test uses $g =$ analytic on every face and therefore does *not* measure the quality of the $g = 0$ absorbing boundary on oblique waves — that is a physics question about absorbing boundaries (Sec. 5.6) and a design question about replacing the SAT with a PML (Sec. 6), not a convergence question about the SAT implementation.

9 References and notes

- Strand, B. (1994), *Summation by parts for finite difference approximations for d/dx*, J. Comput. Phys. 110, 47–67.
- Svärd, M., Nordström, J. (2014), *Review of summation-by-parts schemes for initial-boundary-value problems*, J. Comput. Phys. 268, 17–38. The global-L^2 rate estimate $\min(2p, s+1)$ used in Sec. 8 is from this review.
- Munz, C.-D., Omnes, P., Schneider, R., Sonnendrücker, E., Voss, U. (2000), *Divergence correction techniques for Maxwell solvers based on a hyperbolic model*, J. Comput. Phys. 161, 484–511.
- Bérenger, J.-P. (1994), *A perfectly matched layer for the absorption of electromagnetic waves*, J. Comput. Phys. 114, 185–200. The original split-field PML paper.
- Roden, J. A., Gedney, S. D. (2000), *Convolutional PML (CPML): an efficient FDTD implementation of the CFS-PML for arbitrary media*, Microwave Opt. Technol. Lett. 27, 334–339. The CPML variant recommended for the proposed extension in Sec. 6.

A Summary of SAT formulas

For reference, the full set of SAT contributions applied at the boundary rows, derived in Sec. 3. Let $s = \tau \cdot (H_0^{-1})/h$ be the per-axis penalty scale (with $H_0^{-1} = 48/17$ for the $(4, 2)$ SBP norm), and define c, PP as in Sec. 3.2. Where g is zero, the δ quantities reduce to the evolved fields themselves.

Lower-z (nonzero g, the incoming beam):

$$
\begin{aligned}
\dot{B}_x &\mathrel{-}= s \cdot \tfrac{c}{2}(\delta B_x - PP \cdot \delta D_y) \\
\dot{D}_y &\mathrel{-}= s \cdot \tfrac{c}{2}(-\delta B_x / PP + \delta D_y) \\
\dot{B}_y &\mathrel{-}= s \cdot \tfrac{c}{2}(\delta B_y + PP \cdot \delta D_x) \\
\dot{D}_x &\mathrel{-}= s \cdot \tfrac{c}{2}(\delta B_y / PP + \delta D_x) \\
\dot{B}_z &\mathrel{-}= s \cdot \tfrac{1}{2}(\delta B_z - \delta \Psi_B) \\
\dot{\Psi}_B &\mathrel{-}= s \cdot \tfrac{1}{2}(-\delta B_z + \delta \Psi_B) \\
\dot{D}_z &\mathrel{-}= s \cdot \tfrac{1}{2}(\delta D_z - \delta \Psi_D) \\
\dot{\Psi}_D &\mathrel{-}= s \cdot \tfrac{1}{2}(-\delta D_z + \delta \Psi_D)
\end{aligned}
$$

Upper-z ($g = 0$):

$$
\begin{aligned}
\dot{B}_x &-= s \cdot \tfrac{c}{2}(B_x + PP \cdot D_y) \\
\dot{D}_y &-= s \cdot \tfrac{c}{2}(B_x/PP + D_y) \\
\dot{B}_y &-= s \cdot \tfrac{c}{2}(B_y - PP \cdot D_x) \\
\dot{D}_x &-= s \cdot \tfrac{c}{2}(-B_y/PP + D_x) \\
\dot{B}_z &-= s \cdot \tfrac{1}{2}(B_z + \Psi_B) \\
\dot{\Psi}_B &-= s \cdot \tfrac{1}{2}(B_z + \Psi_B) \\
\dot{D}_z &-= s \cdot \tfrac{1}{2}(D_z + \Psi_D) \\
\dot{\Psi}_D &-= s \cdot \tfrac{1}{2}(D_z + \Psi_D)
\end{aligned}
$$

Lower-x, Lower-y, Upper-x, Upper-y: Obtained from the lower-z and upper-z formulas above by cyclic permutation of the axes. The explicit forms are in `maxwell_eqs.c`, macros `APPLY_SAT_{LOWER,UPPER}_{X,`